

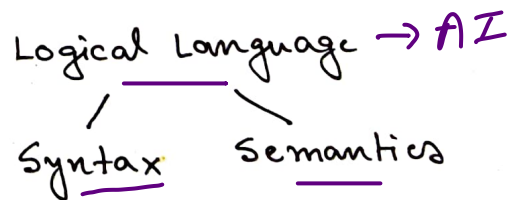
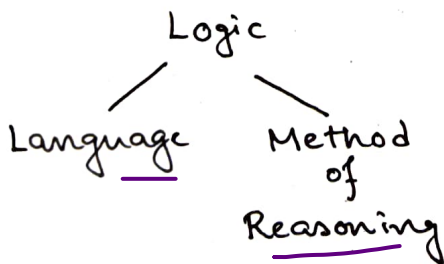
Knowledge Representation

- Knowledge base is an important aspect of intelligent agent. A knowledge base is a set of sentences in a language called the knowledge representation language.
- Logic is the primary vehicle for representing ~~the~~ and reasoning about knowledge. The dealing will be specifically formal logic.

The advantage of using formal logic as a language of AI is that it is precise and definite.

- The severe limitation of logic that it can not represent uncertainty, although the reasoning carried out by human, i.e., handling knowledge is uncertain.

Similarly, natural language reasoning requires inferring hidden state, namely, the intuition.



Syntax — Specifies the symbols in the language and how they can be combined to form sentence.

Semantics - It specifies what fact in the world a sentence refers to. Hence, also specifies how you assign a truth value to a sentence based on its meaning in the world.

Propositional Logic -

In this logic (PL) an user defines a set of propositional symbols, like P & Q .

Ex. - 'P' means 'It is hot'

Q means 'It is humid'

R means 'It is raining'

Example of PL sentences

✓ $(P \wedge Q) \Rightarrow R$: If it is hot and humid, then it is raining

$Q \Rightarrow P$: If it is humid then it is hot

Model - It is an interpretation of a set of sentences such that each sentence is true. A model is just a formal mathematical structure that stands in for the world.

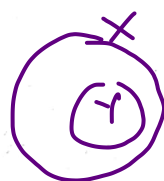
Tautology/valid - A sentence is a valid sentence when that is true under all interpretations.

Inconsistent - The sentence which is false under all interpretation.

Connective is a logical operator that connects simple statements for constructing more complex sentences.

List of connects in propositional logic -

<u>operators</u>	<u>Notations</u>
AND	$\wedge$ ✓
OR	$\vee$ ✓
Negation	$\neg, \sim$ ✓
If P then Q	$P \rightarrow Q$
If P then Q and if Q then P	$P \leftrightarrow Q$ ✓
Implication	$\Rightarrow$ ✓
Bi-directional Implication	$\Leftrightarrow$ ✓
Identity	$\equiv$ ✓
Logical Entailment	$\models$ ✓
Derivability	$\vdash$ ✓



The symbol $X \vdash Y$ implies that Y has been ~~der~~ derived from X .

The logical entailment ~~sub~~ relation ~~$X \vdash Y$~~ $X \models Y$ means that Y logically ~~is~~ follows from X .

List of tautologies in Propositional Logic -

1. $\neg \neg p \equiv p$ ✓

2. $p \wedge q \equiv q \wedge p$

3. $p \vee q$ $\equiv$ $q \vee p$

4. $(p \wedge q) \wedge r$ $\equiv$ $p \wedge (q \wedge r)$

5. $(p \vee q) \vee r$ $\equiv$ $p \vee (q \vee r)$

6. $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$

7. $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

8. $\neg (p \wedge q) \equiv \neg p \vee \neg q$

9. $\neg (p \vee q) \equiv \neg p \wedge \neg q$

10. $p \vee p \equiv p$

11. $p \wedge p \equiv p$

12. $p \wedge q \rightarrow p$

13. $p \wedge q \rightarrow q$

Inference in Propositional Logic -

Inference is a technique by which ^{from} given a set of facts or axioms or premises $F_1, F_2, \dots, F_n$, a goal G is to be derived.

Ex - ✓ "Where there is smoke there is fire" ✓

from this fact :

"There is smoke in the hill"

It can be easily derived

"Then the hill is on fire"

Basic knowledge given

There are three widely used rules -

(i) Modus Ponens;

Given $P \Rightarrow Q$ and P to be true, Q is true

(ii) Modus tollens;

Given $P \Rightarrow Q$ and $\sim Q$ to be true, $\sim P$ is true

(iii) Chain Rule;

Given $P \Rightarrow Q$ and $Q \Rightarrow R$ to be true $P \Rightarrow R$ is true

$$14. p \rightarrow p \vee q$$

$$15. q \rightarrow p \vee q$$

Sematic Method for Theorem Proving-

The following ~~notations~~ notations will be used to ~~be~~ present a symbolic theorem, starting that conclusion 'c' follows from a set of premises $p_1, p_2, p_3 \dots p_n$

$$\underline{p_1, p_2, \dots p_n \Rightarrow c}$$

$$\underline{p_1, p_2, \dots p_n \models c}$$

✓ Construct a truth table representing the relationship of p_1 through p_n with 'c'.

Then the test of validity of the theorem by checking whether both forward and backward chaining methods hold good.

Let,

we define $p_1 = \text{the-sky-is-cloudy}$ ✓

$p_2 = \text{it-will-rain}$ ✓

✓ $p_3 \equiv p_1 \rightarrow p_2$ — Conclusion

} Facts

Three propositions. Now form a truth table of p_1, p_2 & p_3 and then check the forward & back ward chaining for the following theorem

✓ $p_3 = \text{if (the-sky-is-cloudy) then (it-will-rain)}$

p_1 and p_2 above represents premises and conclusion respectively for the if-then clause.

It is obvious from common sense that p_2 directly follows from p_1 and p_3 .

$$p \Rightarrow q \quad \neg q \quad \neg p$$

$$\underline{\neg p \wedge p_2} \quad \text{Theorem 10}$$

$$p_3 \equiv p_1 \rightarrow p_2$$

$$\equiv \neg(p_1 \wedge \neg p_2) \quad \text{Since } p_1 \text{ true and } p_2 \text{ false can not occur together}$$

$$\checkmark \equiv \underline{(\neg p_1 \vee p_2)} \quad \text{— De Morgan's Law}$$

The truth table p_1, p_2 and $p_1 \rightarrow p_2$

p_1	p_2	$p_3 \equiv p_1 \rightarrow p_2$
		$\equiv \neg p_1 \vee p_2$

0 0 1

0 1 1

$\checkmark$ 1 0 0 $\checkmark$ — Backward chaining $\neg$

$\checkmark$ 1 1 1 — Forward chaining $\rightarrow$

Theorem is true

Forward chaining : when all premises are true, check whether the conclusion is true. — under this circumstance ~~for~~ forward chaining holds good.

for example when p_1 & p_3 are true, check p_2 is true or not. For the last row it holds good. Hence forward chaining satisfy.

Backward chaining: When all ^{the} consequences are false, check whether at least one of the premises is false

In Example - $p_2 = 0$ in 1st & 3rd row. Note that when $p_2 = 0$ then $p_1 = 0$ in the first row and p_3 in the third row. So backward chaining hold good.

Syntactic Methods for theorem proving -

Assuming p, q and r to be propositions, the list of the standard theorems is

- | | |
|--|--|
| ✓ 1. $p, q \Rightarrow p \wedge q$ | ✓ 6. $p \rightarrow q, q \rightarrow r \Rightarrow p \rightarrow r$ |
| 2. $p, p \rightarrow q \Rightarrow q$ | 7. $p, p \rightarrow q, q \rightarrow r \Rightarrow r$ |
| 3. $\neg p, p \vee q \Rightarrow q$ | 8. $p \vee (q \wedge \neg q) \Leftrightarrow p$ |
| 4. $\neg p, p \rightarrow q \Rightarrow \neg p$ | 9. $p \wedge (q \vee \neg q) \Leftrightarrow p$ |
| 5. $p \vee q, p \rightarrow r, q \rightarrow r \Rightarrow r$ | 10. <u>$p \rightarrow q \Leftrightarrow \neg p \vee q$</u> |
| 11. $\neg (p \rightarrow q) \Leftrightarrow p \wedge \neg q$ | ✓ 12. $p \leftrightarrow q \Leftrightarrow (p \rightarrow q) \wedge (q \rightarrow p)$ |
| 13. $p \leftrightarrow q \Leftrightarrow (p \wedge q) \vee (\neg p \wedge \neg q)$ | |
| ✓ 14. $p \rightarrow (q \rightarrow r) \Leftrightarrow (p \wedge q) \rightarrow r$ | |
| ✓ 15. $p \rightarrow q \Leftrightarrow \neg q \rightarrow \neg p$ | |

• Prove composition theorem.

It can be stated as follows. When p & q are two propositions, the theorem ~~the~~ takes the form of

$$p \rightarrow q \Leftrightarrow \neg p \vee q$$

Now,

$$\text{L.H.S} \Rightarrow p \rightarrow q$$

$$\Rightarrow \neg p \vee q \quad (\text{by } 10)$$

$$\Rightarrow q \vee \neg p$$

$$\Rightarrow \neg(\neg q) \vee \neg p$$

$$\Rightarrow \neg \cancel{q} \vee \neg \cancel{p}$$

$$7q \rightarrow 7p$$

Substitution Method

$$\neg(q \vee \neg p) \Rightarrow 7q \vee b$$

$$\neg(\underbrace{q}_a \vee \underbrace{\neg p}_b) \Rightarrow 7a \vee b$$

- Since one false & one true cannot occur together

$$7q \rightarrow 7p$$

$$a \rightarrow b$$

$$7a \rightarrow 7b$$

• Prove Theorem 14.

$$\text{L.H.S} \Rightarrow p \rightarrow (q \rightarrow r)$$

$$\Rightarrow \underbrace{p}_a \rightarrow \underbrace{(7q \vee r)}_b \quad [\text{by } 10]$$

$$\Rightarrow 7p \vee (7q \vee r) \quad [\text{by } 10]$$

$$\Rightarrow (7p \vee 7q) \vee r \quad [\text{Since this is a tautology (5)}]$$

$$\Rightarrow \neg(\neg p \wedge \neg q) \vee r \quad [\text{by De Morgan's Law}]$$

$$\underbrace{\neg(\neg p \wedge \neg q)}_a \vee \underbrace{r}_b$$

$$\Rightarrow (p \wedge q) \rightarrow r \quad [\text{by } 10]$$

Hence by syntactic approach for theorem proving can be done in two ways - (i) by methods of substitution and (ii) by Wang's algorithm.

The above two proves are by substitution method. So in method of substitution any of the side ~~is chosen~~ of the statement to be proved is chosen and standard formulas, are applied selectively to prove the other side of the statement.

- Standard theorems 1 to 15. and also

Tautologies are used.

Theorem proving by using Wang's Algorithm

Any theorem ~~is~~ in propositional logic is often represented by

$$p_1, p_2, p_3, \dots, p_n \Rightarrow q_1, q_2, \dots, q_m$$

where p_i & q_j are presenting propositions.

Comma $\rightarrow$ Left side Right side
 AND OR

$$p_1 \wedge p_2 \wedge p_3 \dots \wedge p_n \Rightarrow q_1 \vee q_2 \vee q_3 \dots \vee q_m$$

Algo.

Step I: Starting condition:

Present all sentences, involving only, $\wedge, \vee$ and $\neg$ operators.

Step II: Recursive procedure:

Repeat steps (a), (b) or (c) whichever is appropriate until the stopping condition, present in step III occurs.

(a) Negation Removal:

In case negated term is present at any side bring it ~~to~~ to the other side of implication symbol without its negation symbol.

Ex. $p, q, \neg r \Rightarrow s$

$$\vdash p, q \Rightarrow r, s$$

(b) AND, OR Removal:

If L.H.S contains $\wedge$ operator, replace it by comma. On the other hand if R.H.S contains $\vee$ operator, also replace it by comma.

Ex.
$$p \wedge r, s \Rightarrow s \vee t$$
$$\vdash p, r, s \Rightarrow s, t$$

(c) Theorem splitting:

If L.H.S contain OR operator, then split the theorem into two sub theorems by replacing the OR operator. Alternatively, if the R.H.S contains AND operator, then also split the theorem into two sub-theorems.

Ex.
$$p \vee r \Rightarrow s, t$$

$$\vdash p \Rightarrow s, t \quad \& \quad r \Rightarrow s, t \quad : \text{sub-theorems}$$

$$p, r \Rightarrow s \wedge t$$

$$\vdash p, r \Rightarrow s \quad \& \quad p, r \Rightarrow t \quad : \text{sub-theorems}$$

Step III: Stopping condition:

✓ Stop theorem proving process if either of
(a) or (b) listed below, occurs.

(a) If both L.H.S and R.H.S contain atomic terms, then stop

(b) If L.H.S and R.H.S. have been represented as a collection of atomic terms, separated by commas only and there exist no common terms on both sides, then stop

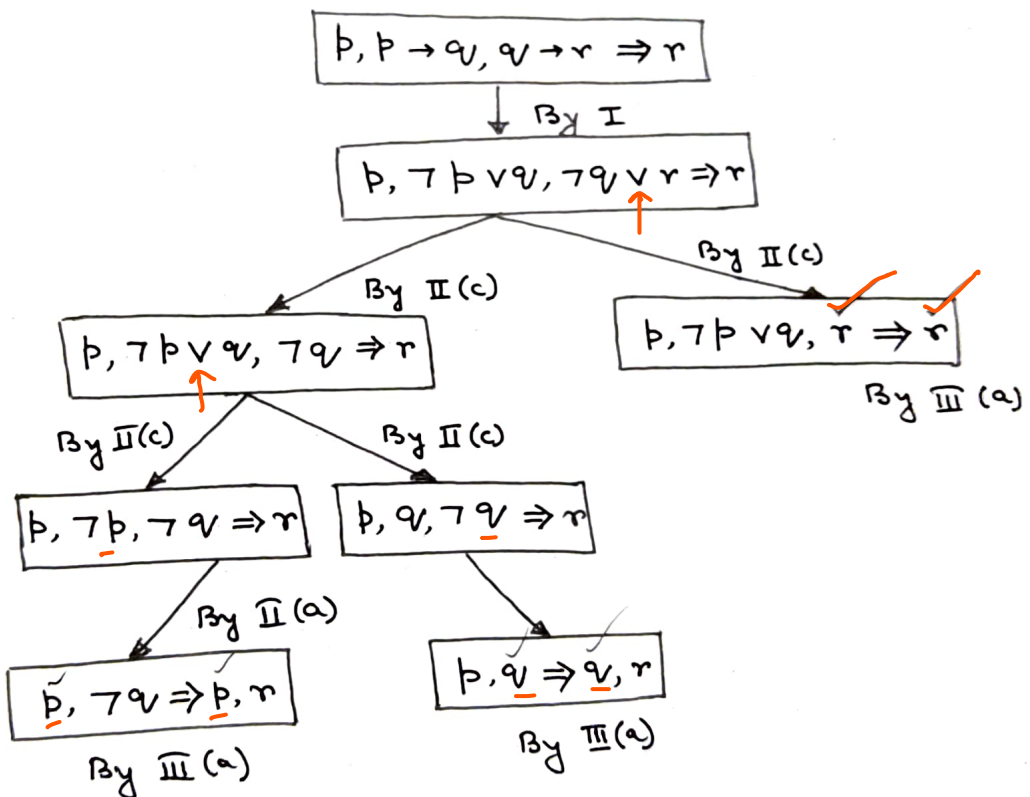
End

- Prove the chaining rule with Modus Ponens using Wang's algorithm.

Proof: the chaining rule with Modus Ponens can be described as

$$p, p \rightarrow q, q \rightarrow r \Rightarrow r$$

A tree can be constructed, where nodes denote every propositions. An arc in the tree ~~node~~ denotes steps of Wang's algorithm.



Since all the terminals of the ~~theorem~~ tree have been stopped by using III (a), the theorem hold good.

Resolution in Propositional Logic -

The principle of resolution theorem for any clause p, q, r ;

$$p \vee r, q \vee \neg r \Rightarrow p \vee q$$

This theorem can be proved by Wang's algorithm.

Algorithm -

Input : A set of clauses, called axioms and a goal

Output : To test whether the goal is derivable from the axioms.

Begin

1. Construct a set S of axioms plus the negated goal
2. Represent each element of S into conjunctive normal form (CNF) by the following steps:
 - a) Replace 'If-then' operator by NEGATION and OR operator by theorem 10
 - b) Bring each modified clause into the following form and then drop AND operators connected between each square bracket. The clauses thus obtained are in conjunctive normal form (CNF).
It may be noted that p_{ij} may be negated or non-negated form.
3. Repeat
 - a) Select any two clauses from S, such that one clause contains one negated literal and the other clause contains its corresponding positive (non-negated literal).

b) Resolve these two clauses and call the resulting clause the resolvent. Remove the parent clause from S

✓ Until a null clause is obtained or no further progress can be made.

4. If a null clause is obtained, then report: "goal is proved."

Consider the following knowledge base:

- ✓ 1. The-humidity-is-high $\vee$ the-sky-is-cloudy
- ✓ 2. If the-sky-is-cloudy then it-will-rain
- ✓ 3. If the-humidity-is-high then it-is-hot
- ✓ 4. It-is-not-hot

and the goal: it will rain.

$$p \rightarrow r \\ 7p \vee r - Th. 10$$

Prove by resolution theorem that the goal is derivable from the knowledge base.

Proof: let us consider following symbols

p = the-humidity-is-high

q = the-sky-is-cloudy

r = it-will-rain

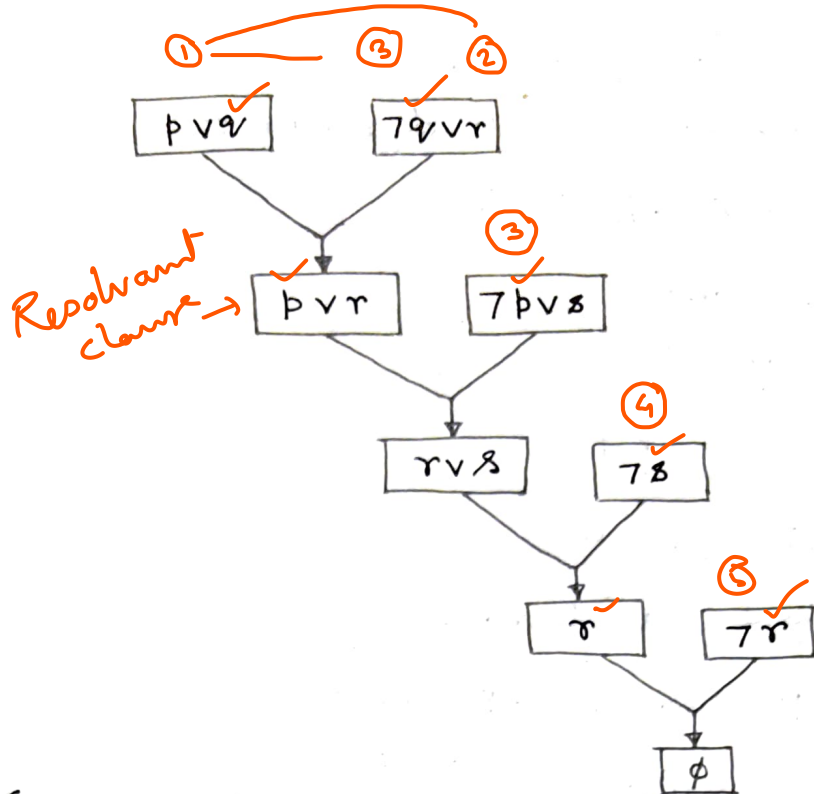
s = it-is-hot

The CNF of the following clauses

- 1. $p \vee q$
- ✓ 2. $7q \vee r$
- ✓ 3. $7p \vee s$
- ✓ 4. $7s$

and the negated goal = $7r$

Thus set S includes all 5 clauses.



Since the termination clause is null, the goal is proved.

Soundness -

A proof process is called sound, if any ^{inference α} ~~inference α~~ has been proved from a set of axioms S by a proof procedure, i.e., $S \vdash \alpha$ it follows logically from S , i.e., $S \models \alpha$

Completeness -

A proof process is called complete, if for any ^{inference} ~~inference~~ α , that follows logically from a given set of axioms, i.e., $S \models \alpha$, the proof procedure can prove α , i.e., $S \vdash \alpha$

In simple words soundness refers to the correctness of the proof procedure, completeness implicates that all the possible inferences can be derived by using the algorithm.

Set of given clauses S and a goal is α
we derive α from S by the resolution theorem.

usual notion is $S \vdash \alpha$.

We need to prove that the derivation is logically sound

—thus $S \models \alpha$

we presume $S \models \neg \alpha$

we assign truth values to all propositions use in α

We now claim that for such assignment, resolution of any two clauses from S will be true.

Thus the resulting clause even after exhaustion of
• all clauses from S ~~will be true~~ through
resolution will not be false.

Thus $S \vdash \alpha$ is a contradiction

Hence assumption $S \models \neg \alpha$ is true.

Examples of Predicate Logic :-

1. Logical representation of all dogs have tails

$$\forall x : \text{dog}(x) \rightarrow \text{has tail}(x)$$

2. Marcus was a man

$$\text{man}(\text{Marcus})$$

3. Marcus was a Pompeian

$$\text{Pompeian}(\text{Marcus})$$

4. All Pompeians were Romans

$$\forall x : \text{Pompeian}(x) \rightarrow \text{Roman}(x)$$

5. Caesar was a ruler

$$\text{ruler}(\text{Caesar})$$

6. All Romans were either loyal to Caesar or hated him.

$$\forall x : \text{Roman}(x) \rightarrow \text{loyal to}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})$$

7. Everyone is loyal to someone

$$\forall x : \rightarrow y : \text{loyal to}(x, y)$$

8. People only try to assassinate rulers they are not loyal to

$$\forall x : \forall y : \text{persons}(x) \wedge \text{ruler}(y) \wedge \text{try to assassinate}(x, y) \rightarrow \neg \text{loyal to}(x, y)$$

An Interface Rule is a rule for obtaining new formula [the consequence] from a set of given formulae [premises].

A theorem T is unsatisfiable if there is no ~~sta~~ structure where all non-logical axioms of T are valid. Otherwise it is satisfiable.

Predicatal Logic — First-order Logic

The capability of reasoning and knowledge representation using predicatal logic is higher than propositional logic.

It includes two more qualifiers

- ~~ess~~ essential qualifier ($\forall$) ✓ — **All**
- existential qualifier ($\exists$) ✓ — **Some**

Ex — Knowledge 1: All boys likes sweets.

$$\forall x (\text{Boy}(x) \rightarrow \text{Likes}(x, \text{sweets})) \quad \checkmark$$

Knowledge 2: Some boys like flying kites.

$$\exists x (\text{Boy}(x)) \rightarrow \text{Likes}(x, \text{Flying-kites})$$

Rewrite the following sentences in FOL (First-order logic)

- S {
1. coconut-crunchy is a biscuit.
 2. Mary is a child who takes coconut-crunchy
 3. John loves children who takes biscuit
 4. John loves Mary. — **Goal**

The above statements can be represented in FOL using two qualifiers —

$x \quad y$

1. Biscuit (coconut - crunchy)
2. child (Mary) $\wedge$ Takes (Mary, coconut - crunchy)
- ✓ 3. $\forall x (\text{child}(x) \wedge \exists y (\text{Takes}(x, y) \wedge \text{Biscuit}(y))) \rightarrow$
Loves (john, x)
4. Loves (john, mary)

Writing a Sentence into clause form -

Algo. -

Step I : Elimination of 'If-then' operator :-
" $\rightarrow$ " operator by ' $\neg$ ' and ' $\vee$ ' operator

Ex. - $\neg \forall x (\neg (\text{child}(x) \wedge \exists y (\text{Takes}(x, y) \wedge \text{Biscuit}(y)))) \vee$
Loves (john, x)

Step II : Reduction of the scope of negation :

Replace ' $\neg$ ' by any of the following

$$a) \neg (P \vee Q) = \neg P \wedge \neg Q$$

$$b) \neg (P \wedge Q) = \neg P \vee \neg Q$$

$$c) \neg (\neg P) = P$$

$$d) \neg (\exists x P) = \forall x \neg P$$

$$e) \neg (\forall x P) = \exists x \neg P$$

Ex - $\forall x (\neg \text{child}(x) \vee \exists y (\neg \text{Takes}(x, y) \vee \neg \text{Biscuit}(y))) \vee$
Loves (john, mary) - (d, e)

Step III : Renaming the variable within the scope of quantifiers :

$\exists x$ by $\exists y$ when $\{x\}$ is a subset of $\{y\}$

Note - In this example x and y are different entity

Step IV: Moving the qualifier in the front of the expression

Ex- $\forall x \forall y \neg \text{child}(x) \vee \neg \text{Takes}(x, y) \vee \neg \text{Biscuit}(y)$
 $\vee \text{Loves}(\text{john}, x)$

Step V: Replacing existential qualifier as skolem function of essential qualifier.

Note: this not applicable as x and y are different entity thus essential qualifier is dropped from sentence

Step VI: Putting the resulting expression conjunctive form (CNF):

Ex- If original expression $P \vee (Q \wedge R)$

then replace it $(P \vee Q) \wedge (P \vee R)$

Step VII: Writing one clause per line.
In this example the expression being in CNF thus no operation is done

Finally - $\text{child}(x), \text{Takes}(x, y), \text{Biscuit}(y) \rightarrow \text{Loves}(\text{john}, x)$

Unification of Predicates -

Two predicates $P(t_1, t_2, t_3 \dots t_n)$ and $Q(s_1, s_2, \dots s_n)$ can be unified if terms t_i can be replaced by s_i or vice-versa.

$\text{Loves}(\text{mary}, y)$ and $\text{Loves}(x, \text{Father}(x))$ can be unified by substitution $S = \{ \text{mary}/x, \text{Father-of}(\text{mary})/y \}$

Conditions for Unification -

- i) Both the predicates to be unified should have an equal ~~most~~ number of terms.
- ii) Neither t_i nor s_i can be a negation operator or predicate or functions of different variables, or if t_i = term belonging to s_i or if s_i = term belonging to t_i then unification is not possible.

Theorem Proving in FOL with Resolution Principle

Suppose, we have to prove a theorem Th from a set of axioms, denoted by $\{A_1, A_2, A_3, \dots, A_n\} \models Th$

Let,

$A_1 = \text{Biscuit}(\text{coconut-crunchy})$

$A_2 = \text{child}(\text{mary}) \wedge \text{Takes}(\text{mary}, \text{coconut-crunchy})$

$A_3 = \forall x (\text{child}(x) \wedge \exists y (\text{Takes}(x, y) \wedge \text{Biscuit}(y))) \rightarrow \text{Loves}(\text{john}, x)$

and $Th = \text{Loves}(\text{john}, \text{mary}) = A_4$

~~Now to prove this theorem we need Robinson's~~

~~Interference Rule.~~

From CNF it can be written as as

✓ $\neg \text{child}(x) \vee \neg \text{takes}(x, y) \vee \neg \text{Biscuit}(y) \vee \text{Loves}(\text{john}, x)$

$\neg \text{Loves}(\text{john}, \text{mary})$

Unification

x/mary

$\neg \text{child}(x) \vee \neg$
 $\text{takes}(x, y) \vee$
 $\neg \text{Biscuit}(y) \vee$
 $\text{Loves}(\text{john}, x)$



$\neg \text{child}(x) \vee \neg \text{takes}(x, y) \vee$
 $\neg \text{Biscuit}(y)$

Unification

Biscuit
(coconut-~~ch~~
crunchy)

y/coconut-
crunchy



$\text{child}(\text{mary})$

$\neg \text{child}(\text{mary}) \vee \neg \text{takes}(\text{mary},$
 $\text{coconut crunchy})$



$\text{takes}(\text{mary},$
 $\text{coconut crunchy})$

$\neg \text{takes}(\text{mary}, \text{coconut crunchy})$



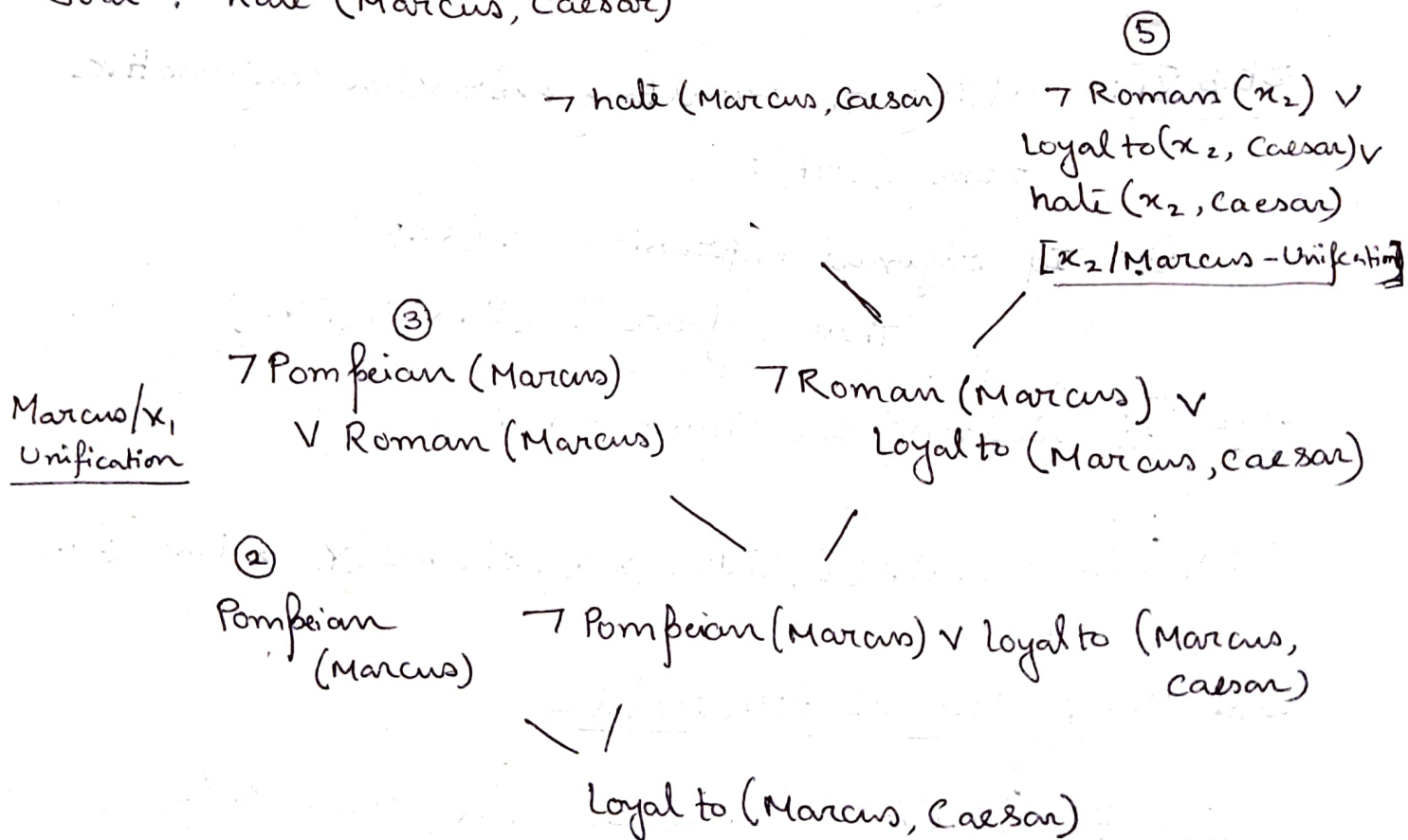
ϕ

Resolution in Predicate Logic

Axioms in clause form

1. Man (Marcus)
2. Pompeian (Marcus)
3. $\neg$ Pompeian (x_1) $\vee$ Roman (x_1)
4. ruler (Caesar)
5. $\neg$ Roman (x_2) $\vee$ loyal to (x_2 , Caesar) $\vee$ hate (x_2 , Caesar)
6. Loyal to (x_3 , fl(x_3))
7. $\neg$ man (x_4) $\vee$ $\neg$ ruler (y_1) $\vee$ $\neg$ byassassinati (x_4 , y_1) $\vee$ loyal to (x_4 , y_1)
8. byassassinati (Marcus, Caesar)

Goal : hate (Marcus, Caesar)



In comparison to propositional logic, the contradiction is more in predicate logic. As $L \wedge \neg L$ is simply contradicting. But $\text{man}(\text{John}) \wedge \neg \text{man}(\text{spot})$ may or may not be contradiction. It is due to the fact that in predicate logic both literal and argument are to be checked. Thus a match procedure is required to compare two literal and discover whether there exist a set of substitution that makes them identical. There is a straight forward recursive procedure called unification.

Skolem function : In mathematical logic, reduction to Skolem form is a method of removing existential quantifiers from formal logic statements.